10. This VGP is trained as follows:

```
num_datapoints = 100 ❶
torch.manual_seed(0) ❶
model = ApproximateGPModel( ❶
    train_x[torch.randint(train_x.shape[0], (num_datapoints,)), :] ❶
) ❶

likelihood = gpytorch.likelihoods.GaussianLikelihood(
    noise_constraint=gpytorch.constraints.GreaterThan(1e-1)
)

train_dataset = torch.utils.data.TensorDataset(train_x, train_y) ❷
train_loader = torch.utils.data.DataLoader( ❷
    train_dataset, ❷
    batch_size=100, ❷
    shuffle=True ❷
) ❷

ngd_optimizer = gpytorch.optim.NGD( ❸
    model.variational_parameters(), num_data=train_y.size(0), lr=0.1 ❸
) ❸

hyperparam_optimizer = torch.optim.Adam( ❹
    [{"params": model.parameters()}, {"params": likelihood.parameters()}], ❹
    lr=0.01 ❹
) ❹

mll = gpytorch.mlls.VariationalELBO(
    likelihood, model, num_data=train_y.size(0)
)

model.train()
likelihood.train()

for i in tqdm(range(10)):
    for x_batch, y_batch in train_loader:
        ngd_optimizer.zero_grad()

        output = model(x_batch)
        loss = -mll(output, y_batch)

        loss.backward()

        ngd_optimizer.step()
        hyperparam_optimizer.step()
```

❶ Randomly pick one hundred points as the initial inducing points.

❷ Prepare the mini batches.

❸ Natural gradient descent for variational parameters.

❹ Adam for the other parameters.

11. On the same Macbook, training took 6 seconds, a 400% speedup.

The solution is included in `CH11/02 - Exercise.ipynb`.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>